Universidade do Algarve | Parallel and Distributed Systems | 2026

HW2: Social Networking in a Distributed Environment

Nome: _____ №:_____

Objetivo. O principal objetivo deste laboratório é que os estudantes se familiarizem com a modelação de arquiteturas de software para sistemas distribuídos (arquitetura cliente-servidor).

1 Antes de começar

(**a**) Abra o Rodin e importe o projeto recitation03.zip que está disponível no Tutoria. Não descomprima o ficheiro zip, mas importe-o como um ficheiro de arquivo.

(**b**) Terá de submeter um projeto Rodin sob a forma de um ficheiro zip como resposta a este laboratório. Não será necessário submeter um ficheiro pdf.

(**c**) O projeto Rodin recitation03.zip inclui duas máquinas: uma máquina abstrata poporo com um modelo simplificado do Facebook e uma máquina poporo_distrib com um modelo para publicação de conteúdo num ambiente distribuído. **Não precisará, nem está autorizado, a editar nenhuma destas duas máquinas**.

2 Máquina abstrata de Poporo

O projeto Rodin inclui duas máquinas: uma máquina abstrata poporo com um modelo simplificado do Facebook e uma máquina poporo_distrib, que é o primeiro refinamento de poporo. A máquina abstrata inclui apenas uma operação (evento), upload, que publica o item de conteúdo c na página da pessoa p. A máquina abstrata não conhece a *mecânica* de como o item de conteúdo c é publicado num **ambiente distribuído**, mas a máquina poporo_distrib conhece. A máquina abstrata conhece o efeito final (resultado) da publicação de um item de conteúdo; por exemplo, a ação content := content ∪ {c} no evento upload da máquina abstrata indica que eventualmente o conteúdo c é adicionado ao conjunto de itens de conteúdo publicados. A máquina abstrata não especifica a sequência de passos para alcançar o estado em que c está incluído no conjunto de conteúdos. A máquina concreta poporo_distrib especifica essa sequência de passos. Numa primeira fase, o evento send_upload faz um pedido para que o conteúdo c seja publicado na página da pessoa p e, numa segunda fase, o evento process_upload processa o pedido para que o conteúdo seja publicado.

A máquina poporo_distrib é apresentada na Figura 1. A variável pages e o evento upload são definidos da mesma forma que no laboratório anterior. O evento upload publica o item de conteúdo c na página da pessoa p. Um item de conteúdo pode ser uma imagem, texto ou vídeo que p pretende publicar.

3 Poporo num ambiente distribuído

A máquina poporo_distrib modela o Poporo (o nosso modelo do Facebook) numa versão geral e sem restrições de um ambiente distribuído. Na Figura 2, a variável content_c mantém os itens de conteúdo que foram primeiro solicitados e depois publicados. O evento send_upload solicita a publicação de um item de conteúdo c e o evento process_upload (se executado com sucesso) publica o conteúdo. O evento send_upload modifica content (uma vez que extends upload) mas não modifica content_c. O evento process_upload modifica content_c mas não modifica content. Portanto, após a execução de send_upload, a variável content contém elementos que ainda não estão em content_c.

A variável pages_c (ver invariante @invr1_1) é a versão concreta da variável pages: pages_c está para pages assim como content_c está para content. A variável pub_req mantém o registo dos pedidos de publicação

```
machine poporo sees ctx1
variables person // persons in the network
    content // content items in the network
    pages // page content per person
invariants
  @inv1 person ⊆ PERSON
  @inv2 content ⊆ CONTENT
  @inv3 pages ∈ content ⇸ person

events
  event INITIALISATION
    then
      @act1 person ≔ ∅
      @act2 content ≔ ∅
      @act3 pages ≔  ∅
  end

  event upload
    any c p
    where
      @grd1 p ∈ person
      @grd3 c ∈ CONTENT\content
    then
      @act1 content ≔ content ∪ {c}
      @act2 pages ≔ pages ∪ {c↦p}
  end
end
```

Figure 1: Ambiente Rodin IDE

```
machine poporo_distrib refines poporo  sees ctx1

variables person // persons in the network
          content // content items in the net
          pages // page content per person
          pub_req // publishing request
          content_c // concrete version of set content
          pages_c // concrete version of pages

invariants
  @invr1_0 content_c ⊆ CONTENT // concrete version of content
  @invr1_1 pages_c ∈ content_c ↔ person // concrete version of pages
  @invr1_2 pub_req ∈ CONTENT ⇸ person // pending content per person who publishes it
  @invr1_3 dom(pub_req) ∩ content_c = ∅ // pending content items are not yet published
  // two gluing invariants
  @gluer11 content = content_c ∪ dom(pub_req) // it relates content_c to content
  @gluer12 pages = pages_c ∪ pub_req // it relates pages_c to pages
```

Figure 2: Ambiente Rodin IDE

e do servidor para o qual o pedido foi enviado (ver invariante @invr1_2). O invariante @invr1_3 afirma que os itens em content_c ainda não foram publicados.

Existe um conceito fundamental em Event-B chamado **invariante de ligação** (gluing invariant). Um invariante de ligação relaciona o estado da máquina concreta com o estado da máquina abstrata, garantindo que a máquina concreta não é independente nem não relacionada com a máquina abstrata. A máquina concreta deve comportar-se de acordo com o comportamento da máquina abstrata. A máquina concreta poporo_distrib declara dois invariantes de ligação. O invariante @gluer11 relaciona content_c com content. O invariante @gluer12 relaciona pages_c com pages. O invariante @gluer11 afirma que os itens de conteúdo que ainda não foram publicados estão em dom(pub_req). O invariante @gluer12 afirma que as páginas estão atualizadas ou que as últimas alterações estão armazenadas em pub_req.

4 Este laboratório

Observe que não existe qualquer suposição sobre a forma como os pedidos são tratados, em particular, nada é dito sobre a arquitetura do Poporo. Os pedidos podem ser tratados por um servidor que decide se o conteúdo é publicado na rede. Observe também que, por exemplo, content representa o estado final do conteúdo da rede num ambiente distribuído, enquanto content_c representa um estado intermédio. O mesmo se aplica a pages_c e pages.

Neste laboratório, **irá criar um modelo Event-B para publicação de conteúdo num ambiente distribuído assumindo uma arquitetura cliente-servidor**. Não deve editar as máquinas poporo ou poporo_distrib. A máquina poporo_distrib servirá como referência para escrever o seu modelo. Certifique-se de que todas as obrigações de prova são resolvidas. O número de obrigações de prova não resolvidas é uma boa medida da sua classificação. Certifique-se de que instalou o plugin SMT (ver laboratório anterior) para descarregar todas as obrigações de prova. **Não é permitido utilizar expressões quantificadas**.

(**a**) Abra o Rodin versão 3.5 e importe o ficheiro recitation03.zip.
(**b**) Crie um novo contexto que estenda ctx1. Clique com o botão direito em ctx1, selecione *Extend*, e dê o nome ctx2. O contexto ctx2 declarará o conjunto portador SERVER.
(**c**) Crie uma máquina poporo_clientserver como refinamento da máquina poporo.
(**d**) Adicione as seguintes variáveis: server, content_c, pages_c, pub_req, pub_ack, pending_content, e pending_pages.
(**e**) Adicione invariantes que relacionem as variáveis, incluindo dois invariantes de ligação.
(**f**) Adicione eventos para solicitar publicação e para aceitação pelo servidor.

5 O que submeter

Deve submeter um ficheiro zip com o projeto Rodin. Renomeie o projeto para recitation03_a666666 e exporte como ficheiro de arquivo.

6 Avaliação

(**i**) O seu projeto Rodin corresponde a **20**% da nota total, e os restantes **80%** correspondem à defesa oral do seu trabalho de casa. A defesa avaliará a sua compreensão do modelo, as suas decisões de conceção e a sua capacidade de explicar e justificar a sua solução

(**ii**) A nota do projeto Rodin (**20**%) será baseada nos seguintes critérios:

- **Completude.** The model includes all required components of the client-server architecture and all required functionality.
- **Correção formal.** Todas as obrigações de prova estão resolvidas.
- **Correção do refinamento.** A máquina concreta refina corretamente a máquina abstrata, e os invariantes de ligação relacionam adequadamente as variáveis abstratas e concretas.
- **Consistência.** O modelo representa uma arquitetura cliente-servidor (servidores, pedidos, confirmações) e não modela uma arquitetura diferente.
- **Qualidade da modelação.** O modelo está bem estruturado, é legível e utiliza variáveis, invariantes e eventos significativos, sem complexidade desnecessária.

(**iii**) **A defesa oral (80%) avaliará a sua compreensão do modelo. Deve ser capaz de explicar o propósito de cada variável, invariante, invariante de ligação e evento, justificar as suas decisões de modelação e explicar por que razão o seu modelo está correto. As defesas terão lugar na semana de 9 de março durante as respetivas PLs em que os estudantes estão inscritos.**